Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 7:

- Helper functions
- Recursion



A CSR Initiative by



NPTEL

Recall: num_days

```
1    def num_days_helper(month: int, is_leap: bool) -> int:
2        if not(1 <= month <= 12):
3            return 0
4        if month == 2:
5            return 29 if is_leap else 28
6        if (month == 4) or (month == 6) or (month == 9) or (month == 11):
7            return 30
8        return 31
```

- Our "client" changes the signature:

```
def num_days(month: int, year: int) -> int:
```

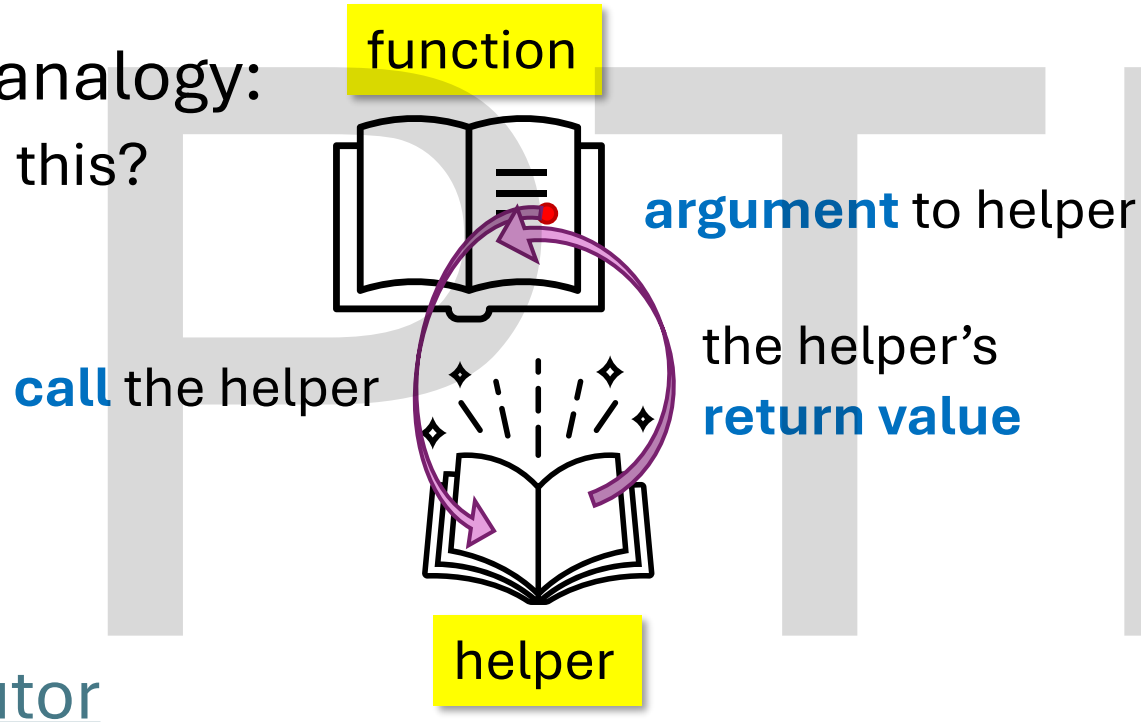


1. Rename our original num_days function as num_days_helper
2. Call the **helper function** in the new function

NPTEL

Helper Functions: Data and Control Flow

- A useful analogy:
  - What's this?



- PythonTutor

NPTEL

Refute Our Friend's Function

```
def num_days(month: int, year: int) -> int:
    is_leap_year = (year % 4 == 0)
    return num_days_helper(month, is_leap_year)
```

Which of the following statements is **FALSE**?

A) If **`year`** is a leap year, then **`year`** is divisible by 4

B) If **`year`** is divisible by 4, then **`year`** is a leap year

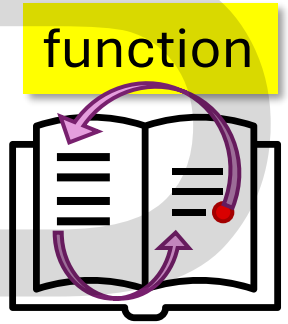
C) The year **`1900`** is a leap year

D) The year **`2000`** is a leap year

NPTEL

Recursive Functions: Self-help!

- A useful analogy:
  - What's this?



- *Example*: For any natural number n, define:

$\text{sum_to}(n)$

$= (1 + 2 + \cdots + n - 1) + n$

$= \text{sum_to}(n - 1) + n$

```
def sum_to(n: int) -> int:
    return sum_to(n-1) + n
```

PythonTutor

NPTEL

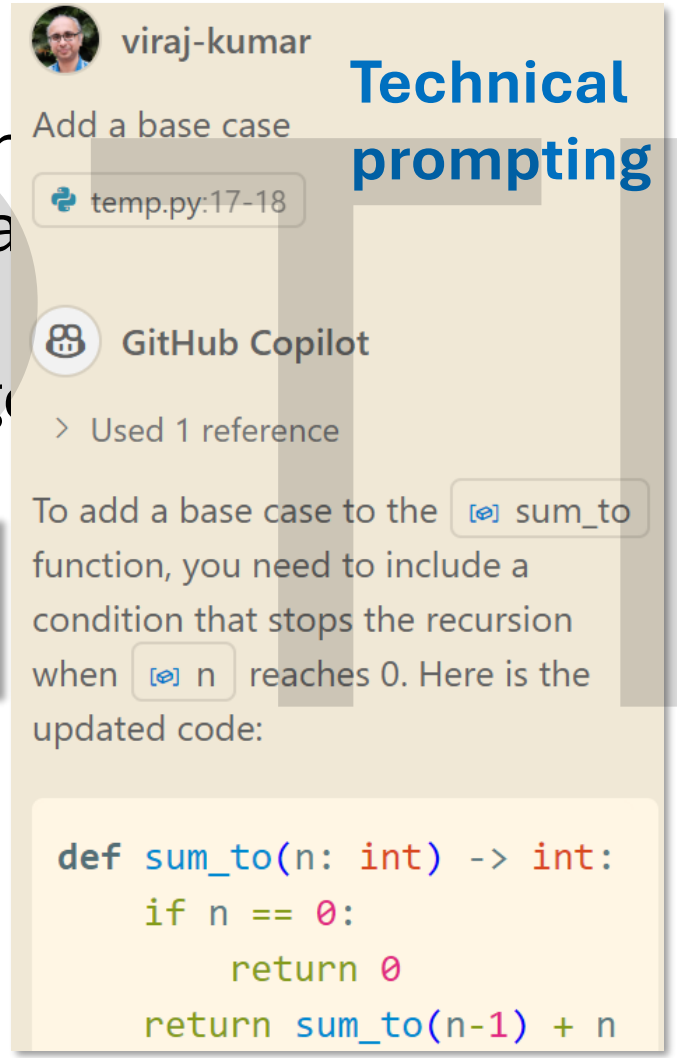
Infinite Recursion

- To avoid infinite recursion, function must have at lea
  - Must be solved *directly*
  - Analogy: No previous pag

```
def sum_to(n: int) -> int:
    return sum_to(n-1) + n
```

When `n == 0`, solve the problem *directly*!

PythonTutor



Technical prompting

NPTEL

Refute Our Friend's Function

```
def sum_to(n: int) -> int:
    """Calculate 1 + 2 + ... + n.
    >>> sum_to(4)
    10
    >>> sum_to(-1)
    0
    """
    return (n * (n + 1)) // 2
```

CodeCheck

NPTEL

Summary

- We can decompose complex problems into simpler problems
- We write **helper functions** to solve the simpler problems, and solve our complex problem by calling these helper functions
- A *special case*: **recursion**
  - The function is its **own helper**!
  - Solved *directly* in the **base case**(s)
  - Otherwise, **infinite recursion**

NPTEL